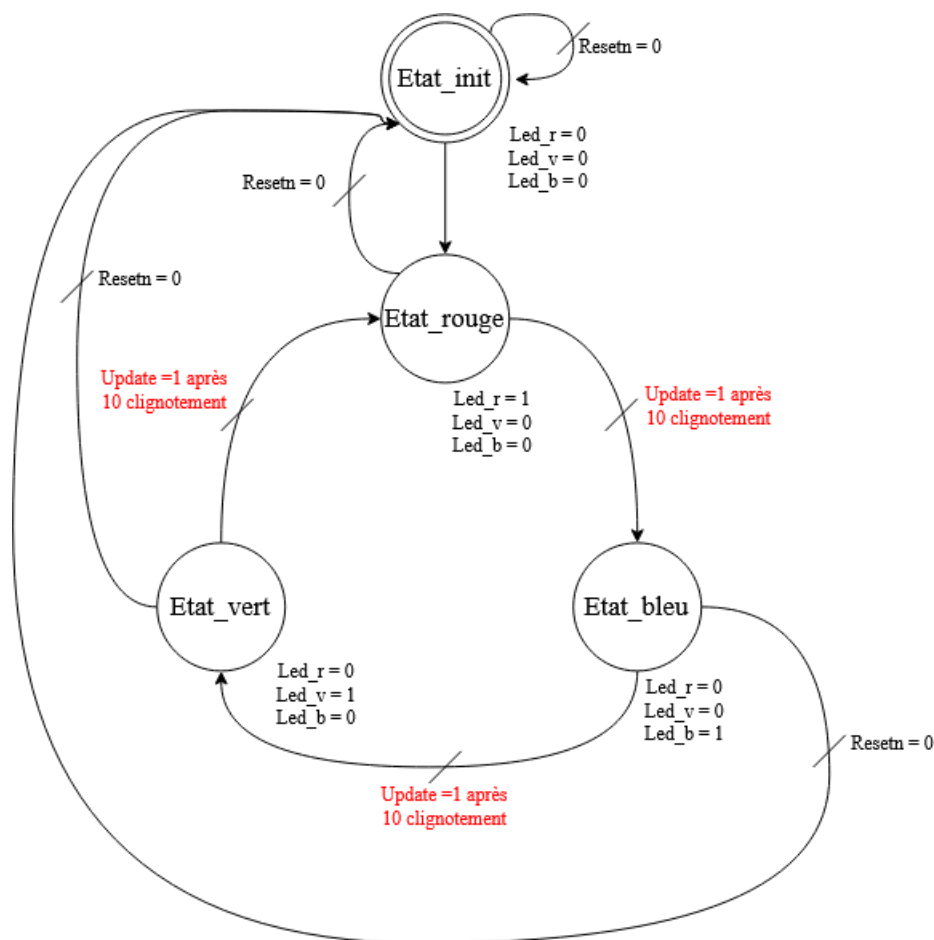


07/08/2026 TP5 : Cross Clock Domain.

Schéma RTL
Résultat de simulation avec chronogramme commentés
Vos résultats de synthèse (analyse de vos ressources)
Vos résultats de STA (analyse du rapport de timing)
Une démonstration de votre design

1. A l'aide du module LED_driver du TP4, créez une architecture RTL permettant de piloter les deux LED RGB. Les LEDs RGB devront clignoter 10 fois en rouge puis 10 fois en bleu et 10 fois en vert avant de recommencer à partir du rouge. En entrée des modules LED_driver le signal update ne devra pas être à 1 pendant plus d'un coup d'horloge. Il doit s'agir d'une impulsion. Le compteur de temporisation devra compter 100 000 000 coups d'horloge (pour une fréquence d'horloge à 100MHz, cela correspond à 1s). Dans la suite de ce TP, **la valeur du compteur de temporisation ne sera pas modifiée, même lorsque les fréquences d'horloges changeront.**



Rq : Indiquer le reset dans la machine à état est facultatif.

Répondre à la problématique proposée implique de réaliser une machine à état avec 4 états, le premier état est l'état initial, toutes les leds sont éteintes. Lorsque *reset* est inactif, on passe dans l'état rouge qui fait clignoter 10 fois la led rouge. Le signal permettant de passer dans l'état suivant est le signal *update* qui sera mis à l'état haut pendant un coup d'horloge au terme de 20 temporisation du compteur de temporisation interne au driver de led.

Led_driver à son propre compteur de temporisation qui compte 100 000 000 coups d'horloges avant de passer de l'état *led_off* à *led_on*, je souhaite connaître le nombre de temporisation afin de changer la couleur au bout de 20 temporisations (10 clignotements).

Il est possible soit, de faire sortir le signal de fin de temporisation de led driver, soit d'instancier un autre compteur. J'ai préféré ajouter le signal de fin de temporisation dans le module *led_driver* afin d'avoir une seule unité de temporisation.

Une représentation RTL est disponible en PNG dans le dossier de rendu, cf TP5_Schema_RTL_fsm_led_driver on y retrouve :

- La machine à État en rouge permettant de changer la couleur des diodes par l'intermédiaire du vecteur de commande *color_code*
- Le compteur de cycle en bleu permettant le générer le signal d'actualisation de *led_driver*.
- Une bascule de « temporisation » en blanc permettant de décaler la prise en compte du signal *d'Update* afin de laisser le temps à la machine à état d'actualiser la valeur de *color_code* à prendre en compte.

J'ai déterminé les équations logiques des changements d'état de la façon suivante :

Current_state [1]	Current_state [0]	Next_state [1]	Next_state [0]
0	0	0	1
0	1	1	0
1	0	1	1
1	1	0	1

$\text{Next_state [0]} = [(\text{Current_state[1]} \text{ ET } \text{Current_state[0]}) \text{ OU } \text{Current_state[1]}]$			
$\text{Next_state [1]} = [(\text{Current_state[1]} \text{ ET } \text{Current_state[0]}) \text{ OU } (\text{Current_state[1]} \text{ ET } \text{Current_state[0]})]$			

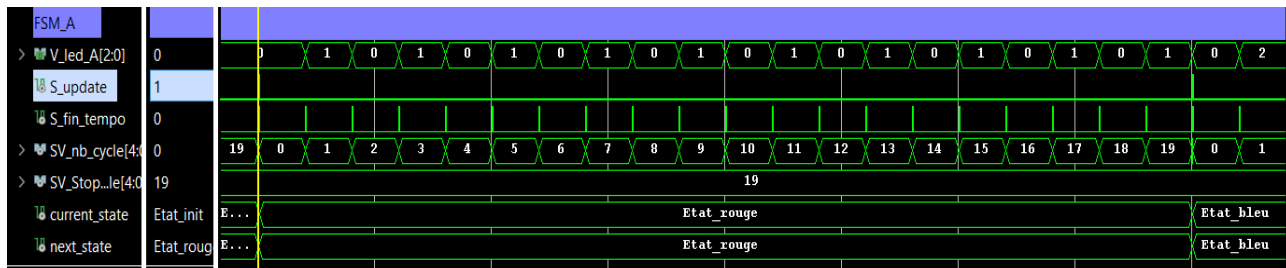
2. Rédigez le code VHDL correspondant à votre architecture.

Signaux TOP	
<i>clk_A</i>	Signal d'horloge principal, commun à une unité de pilotage de led et à la logique générale.
<i>clk_B</i>	Signal d'horloge secondaire pilotant une unité <i>led_driver</i> .
<i>reset_general</i>	L'état haut est l'état actif. Reset est Asynchrone
<i>Restart_general</i>	L'état haut est l'état actif. Restart est Synchrone.
<i>V_led_A</i>	Vecteur 3 bits de pilotage des leds, chaque bit est associé à l'état d'une led. <i>V_led_A</i> sera associé à l'horloge <i>clk_A</i>
<i>V_led_B</i>	Idem, <i>V_led_B</i> sera associé à l'horloge <i>clk_B</i>
Signaux FSM	
<i>S_reset</i>	Signal associé au bouton <i>reset</i> qui sera inversé à l'aide d'une porte not.
<i>S_resetrn</i>	Reset not.
<i>S_update</i>	Signal de mis à jour des drivers permettant la prise en compte de <i>color_code</i> . Signal impulsionnel.
<i>S_update_decal</i>	Idem décalé d'un coup d'horloge à l'aide d'une bascule pour des raisons fonctionnelles.
<i>SV_led</i>	Signal de pilotage des leds.
<i>S_color_code</i>	Vecteur 2 bit de pilotage de la couleur des leds des composants <i>led_driver</i>
<i>S_fin_tempo</i>	Signal de sortie du composant <i>Led_driver</i> indiquant une fin de temporisation. Deux <i>S_fin_tempo</i> indiquent que les leds ont clignotées une seule fois.
<i>SV_Stop_cycle</i>	Vecteur permettant de choisir le nombre de clignotement d'une led avant un changement de couleur.
<i>SV_nb_cycle</i>	Vecteur incrémental permettant de compter le nombre de clignotement.
<i>current_state</i>	Etat de la machine à état, un état pour chaque couleur.
<i>next_state</i>	Etat à venir, <i>Next_state</i> devient <i>Current_state</i> sur <i>Update</i> à l'état haut.

A/ Vérification du bon fonctionnement du composant fsm_led_driver

Élément testé	Critère de succès	Ok/Nok
Signal <i>Update</i>	<i>Update</i> est une impulsion qui déclenche le changement d'état de la machine à état	Ok
Gestion des clignotements	Incrément de <i>Nb_cycle</i> à chaque fin de temporisation. <i>SV_Nb_cycle</i> = 0 lorsque <i>Nb_cycle</i> > <i>SV_stop_cycle</i> .	Ok
Gestion du <i>reset</i>	<i>Nb_cycle</i> vaut 0, <i>Current</i> et <i>Next_state</i> reviennent à l'état initial.	Ok
Gestion du restart	<i>Nb_cycle</i> vaut 0	

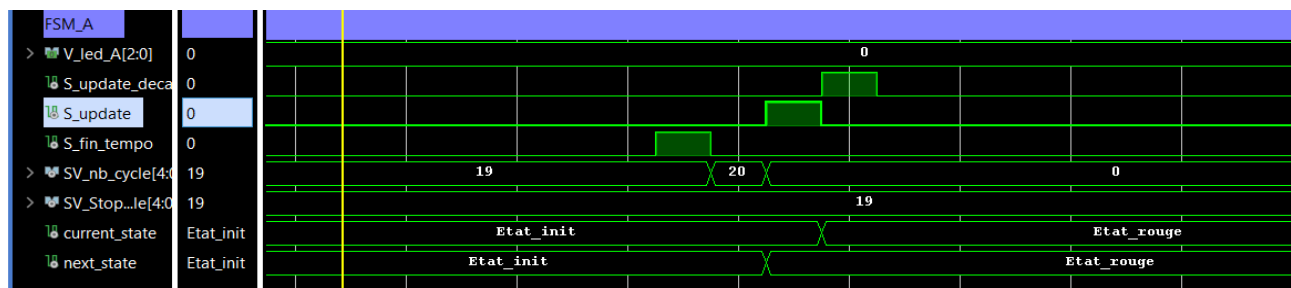
	<i>Next_state vaut l'état rouge.</i>	
Changement d'état	Succession des état : init → rouge → bleu → vert → rouge. Celle-ci intervient au bout de 20 temporisations.	Ok



On constate qu'à chaque état haut de *S_fin_tempo*, la valeur de *SV_nb_cycle* est incrémentée. Ce signal est réinitialisé lorsque 20 temporisations ont été atteinte (Ou 10 clignotements de led).

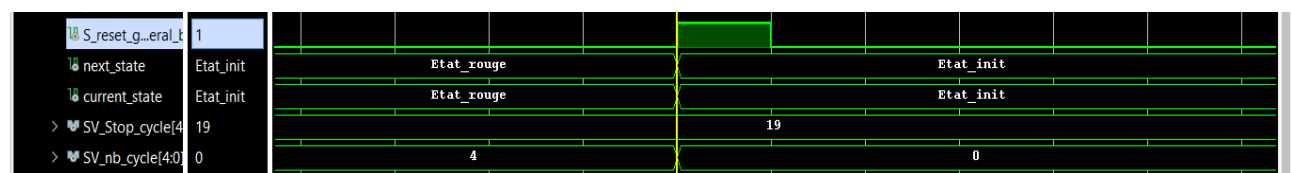
Le signal *d'Update* est bien une impulsion.

V_led, le vecteur de pilotage des leds vaut « 001 », « 010 » ou « 100 » respectivement en binaire (1,2, et 4) en fonction de la led choisie. Ce vecteur est passé à 0 à chaque état haut de *S_fin_tempo* pour réaliser le clignotement.



Zoom sur la partie changement d'État.

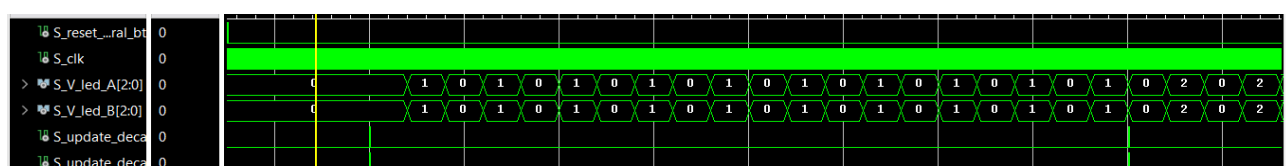
Une bascule a été introduite pour permettre l'actualisation de la machine à état avant de charger *color_code* dans *led_driver*. Le changement d'état intervient lorsque *SV_nb_cycle* > *SV_stop_cycle*.

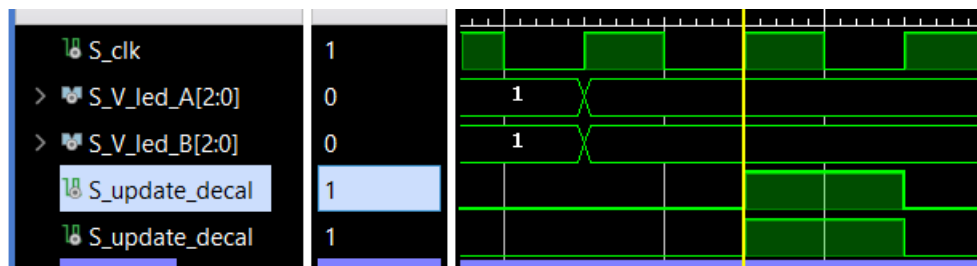


Test du reset.

Sur *reset* actif, *Current_state* et *Next_state* deviennent *Etat_init*, *SV_nb_cycle* est remis à 0.

3. Rédigez le testbench et simulez votre design. Vérifiez que les modules réceptionnent correctement le signal *update*.





Le signal *Update* est correctement reçu par les deux drivers, C'est un signal impulsionnel d'un coup d'horloge.

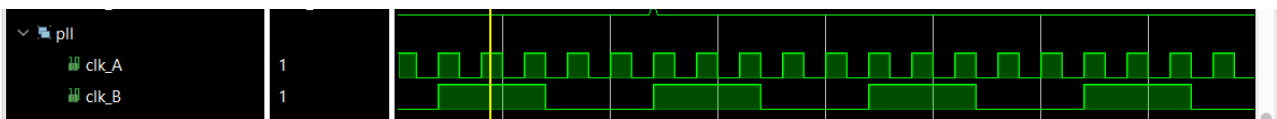
Sur état haut de *restart_general*, *Current_state* et *Next_state* deviennent *Etat_rouge*, *SV_nb_cycle* est remis à 0. Cela fonctionne en implémentation mais pas en simulation. J'ignore pourquoi en simulation, la PLL est remise à zéro sur restart actif alors qu'elle n'y est pas connectée.

4. Modifiez votre design pour gérer deux signaux d'horloge différents. La première horloge, *clkA*, est associé à la logique en dehors des modules *LED_driver* et au module *LED_driver* de la LED0. La deuxième horloge, *clkB*, est associé au module *LED_driver* de la LED1. Le changement de couleur des deux LEDs à lieu lorsque la LED0 à clignoté 10 fois

En première intention, j'ai écrit une composant *Fsm_led_driver* contenant la machine d'État. J'instanciais deux fois ce composant pour gérer le clignotement à des fréquences différentes bien que fonctionnelle, cette solution ne permettait pas démontrer les problèmes relatifs au Cross clock domain.

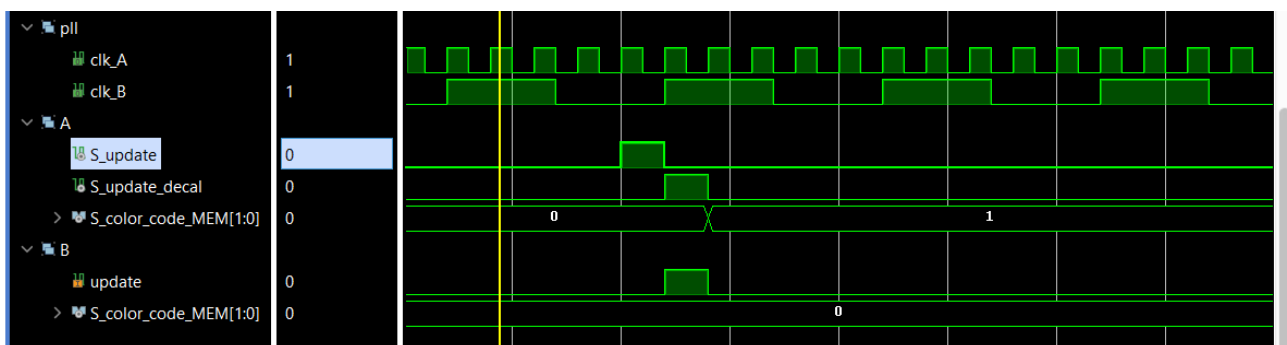
Cf : Schéma RTL « TP5_Q4 » dans le dossier de rendu.

5. Modifiez votre testbench tel que l'horloge *clkA* ait une fréquence de 250Mz et *clkB* 50MHz



Les deux horloges sont bien à des fréquences différentes, $f_{clk_a} > f_{clk_b}$.

6. Lancer une simulation. Que se passe-t-il au niveau des signaux *update* des modules *LED_driver* ? Qu'elle incidence cela a-t-il sur les LEDs ?



On constate que le signal *d'update* n'est pas pris en compte par le driver led piloté par l'horloge la plus lente. Ce signal est synchrone sur le front montant de l'horloge B mais retombe à 0 avant la fin de la première demi-période.

7. Proposez une solution pour corriger le problème lié au changement de domaine d'horloge, vous pourrez vous aider du lien <https://nandland.com/lesson-14-crossing-clock-domains/>.

Pour passer d'un domaine de fréquence **lent vers un rapide**, il faut utiliser deux registres pour réaliser une simple **détection de front montant**.

Lors du passage d'un domaine **rapide vers un domaine lent**, il faut **étirer** les signaux pour permettre à l'horloge de moindre fréquence d'échantillonner les signaux.

Une **fifo** est un outils utile pour envoyer des données vers une domaine de fréquence à un autre.

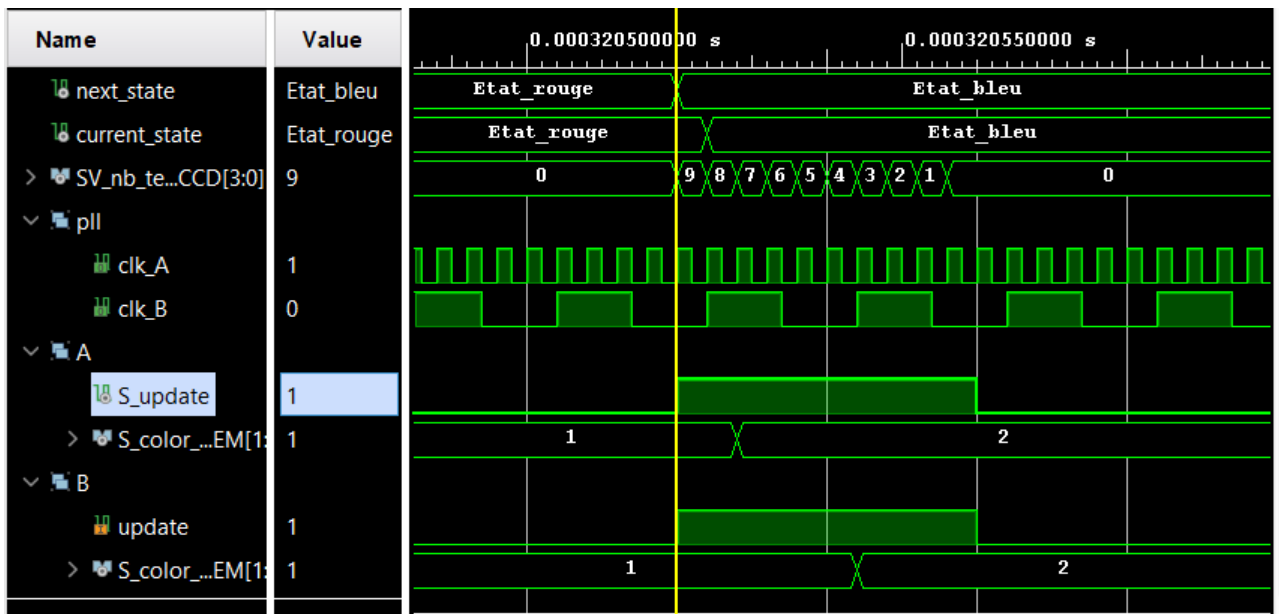
Dans notre cas, nous devons étendre le signal *d'update* afin qu'il soit prise en compte par l'horloge B, le signal doit être étendu d'au moins deux périodes de l'horloge la plus lente soit 10 périodes de l'horloge la plus rapide.

$$T_{tempo} = 2 \times t_B = n \times t_A \text{ donc } n = \frac{2 \times F_A}{F_B} = \frac{2 \times 250 \times 10^6}{50 \times 10^6} = 10$$

Nous implémenterons une variable décomptant de 10 à 0, au terme de laquelle le signal *S_update* est mis à 0. Cette variable s'appelle : *SV_nb_tempo_CDD*, elle est initialisée à 10 lorsque le signal *d'update* est reçu qui dé-incrémente jusqu'à 0.

cf Schéma « RTL TP5 Q7 »

8. Mettez en place votre solution et testez-la en simulation. Si votre résultat de simulation n'est toujours pas valide, proposez une autre solution.



S_update_B est étiré de 10 coups d'horloges, S_color_MEM , le signal mémorisant la couleur de la diode allumée est bien initialisé pour les deux drivers. La solution semble donc fonctionner.

9. Pour générer plusieurs horloges vous aurez besoin d'une PLL. Trouvez quel est le nom de l'IP PLL de l'IP Catalog de Vivado puis ajoutez là à votre architecture. La fréquence de l'horloge en entrée de la PLL est de 100MHz. Les deux horloges en sortie doivent être à 50MHz et 250MHz

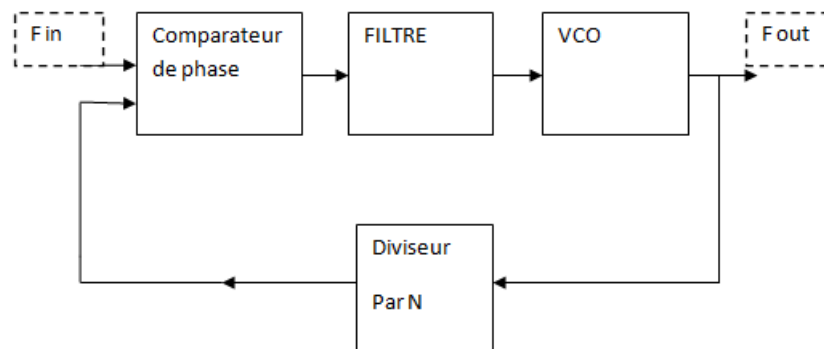
La pll se trouve dans l'IP manager : FPGA Features and Design/Clocking/Clocking Wizard

Qu'est ce qu'une PLL (Phase locked loop) ou boucle à verrouillage de phase ?

Une PLL est constituée d'un **comparateur de phase** suivi d'un **filtre**, d'un **oscillateur** contrôlé en tension (VCO) et d'un **diviseur**. Ces éléments créent grâce à une **boucle de rétroaction** un asservissement ; la phase du signal de sortie est comparé avec celle du signal d'entrée afin de piloter le VCO avec une consigne.

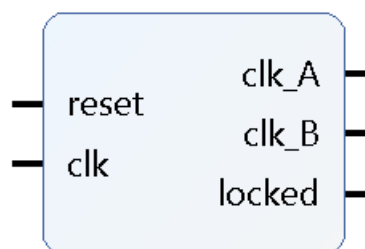
Cet élément peut être utilisé pour démoduler un signal, récupérer une porteuse ou comme dans notre cas de synthétiser des fréquences.

BOUCLE A VERROUILLAGE DE PHASE, principe de base :



La fréquence du quartz de la CORA Z7 est égale à **125MHz**, cette fréquence n'est pas modifiable. Elle sera utilisée pour l'horloge d'entrée de la PLL.

- The Cora Z7 provides a 50 MHz clock to the Zynq PS_CLK input, which is used to generate the clocks for each of the Processing System (PS) subsystems
- Additionally, the Cora Z7 provides an external 125 MHz reference clock directly to pin H16 of the PL



Bloc instancié dans le programme

Nom du signal	Propriété du signal	Remarque
<i>Clk</i>	Input port	Signal d'horloge d'entrée. La fréquence du signal de la carte, fixée par le quartz est égale à 125MHz
<i>Clk_A</i>	Output port	Signal d'horloge à 250MHz
<i>Clk_B</i>	Output port	Signal d'horloge à 50MHz
<i>reset</i>	Input port	
<i>locked</i>	Output port	Indication of the stability of the output signals

Clocking Options | Output Clocks | Port Renaming | PLL2 Settings | Summary

Primitive

☐ MMCM ☒ PLL

Clocking Features **Jitter Optimization**

☒ Frequency Synthesis ☐ Minimize Power ☒ Balanced

☒ Phase Alignment ☐ Minimize Output Jitter

☐ Dynamic Reconfig ☐ Maximize Input Jitter filtering

☐ Safe Clock Startup

Dynamic Reconfig Interface Options

☒ AXI4Lite ☐ DRP ☐ Phase Duty Cycle Config ☐ Write DRP registers

Input Clock Information

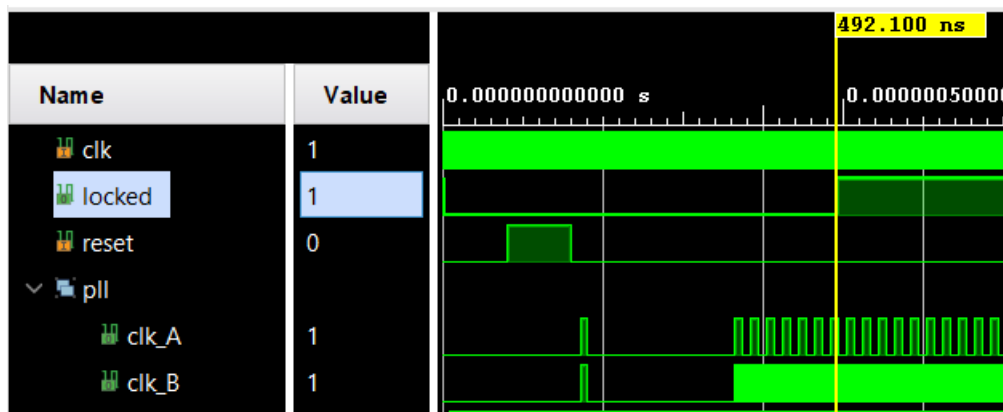
	Input Clock	Port Name	Input Frequency(MHz)		Jitter Options	Input Jitter	Source
☒	Primary	clk	125.000	19.000 - 800.000	UI	0.010	Single e
☐	Secondary	clk_in2	100.000	100.000 - 200.000		0.010	Single e

Output Clock	Port Name	Output Freq (MHz)		Phase (degrees)		Duty Cycle (%)
		Requested	Actual	Requested	Actual	Requested
☒ clk_out1	clk_A	200.000	200.00000	0.000	0.000	50.000
☒ clk_out2	clk_B	50.000	50.00000	0.000	0.000	50.000

On trouve dans la documentation plusieurs recommandations d'utilisation de cet élément :

- **Do not leave any inputs floating.** Relying on synthesis or other optimization tools to tie off the floating values is not recommended, because the values might be different than expected.
- **Connect RST to the user logic,** so that it can be asserted by logic controlled by a reliable clocking source. **Grounding of RST** can cause problems if the clock is interrupted.
- **Use LOCKED output** in the implementation of reset. For example, hold the synchronous logic clocked from the PLL in reset until LOCKED is asserted. The LOCKED signal must be synchronized before it is used in a synchronous

Le signal *locked* est donc utilisé pour maintenir la machine à état dans l'état *d'init* tant que l'horloge n'est pas stable. il y a un temps non négligeable d'établissement des horloges.



Mis en évidence du temps d'établissement des horloges générées par la PLL

Schéma RTL : CF « TP5_Q9.png »

10. Effectuez la synthèse et placez des sondes (ILA) sur les signaux pertinents pour vérifier que le changement de domaine d'horloge s'est correctement passé.

Il n'est pas possible d'afficher les horloges pour les comparer : Placer un ILA sur un signal d'horloge implique d'ajouter de la logique ce qui provoque des retards et donc des dysfonctionnements (Pour rappel, l'horloge à son propre autoroute pour minimiser les délais de propagation, la Clock-tree).

Setup time: The minimum time before the clock edge that data must be stable at the input of a flip-flop or register.

Hold time: The minimum time after the clock edge that data must remain stable.

Clock-to-output delay: The time it takes for the output of a flip-flop to change after the clock edge.

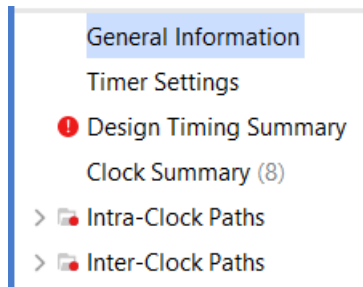
Propagation delay: The time it takes for a signal to travel through combinational logic.

Inter-clock path : The Inter-Clock Paths section of the Timing Summary Report shows the worst slack and total violations for timing paths between **different source and destination clocks**. → look for false clock path (Clock that are asynchronous or identical)

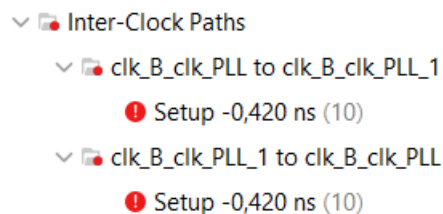
Intra clock path : The Intra-Clock Paths section of the Timing Summary Report displays the worst slack and total violations for timing paths that **share the same source and destination clock**.

(Lorsque j’ai fait les captures les horloge A et B avaient été inversé pour différents tests, dans ce qui suit, c’est l’horloge B qui à la fréquence la plus haute).

Lors de la programmation, nous avons rencontré deux problèmes de timing :



Le premier problème est un problème d’**interclock path** qui est une simple erreur d’interprétation du logiciel car la même horloge porte deux noms différents, le logiciel la interprété comme deux horloges distinctes



Inter-Clock Paths - clk_A_clk_PLL to clk_A_clk_PLL_1

From Clock: clk_A_clk_PLL

To Clock: clk_A_clk_PLL_1

Statistics

Type	Worst Slack	Total Violation	Failing Endpoints	Total Endpoints
Setup	-0,384 ns	-2,203 ns	12	61
Hold	0,105 ns	0,000 ns	0	61

Éditer les contraintes de la façon suivante permet d’éviter les erreurs liées à l’inter-clock path.

1 – Désactiver le fichier de contrainte généré par l’IP PLL

```
# Connect to input port when clock capable pin is selected for input
create_clock -period 8.000 [get_ports clk_pll]
set_input_jitter [get_clocks -of_objects [get_ports clk_pll]] 0.08

set_property PHASESHIFT_MODE WAVEFORM [get_cells -hierarchical *adv*]
```

Contenu du fichier Clk_pll.xdc

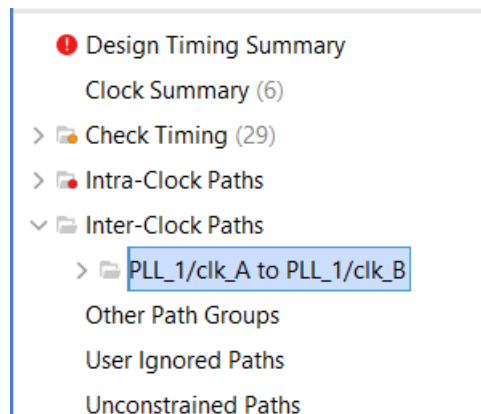
Create Clock									
Position	Clock Name	Period (ns)	Rise At (ns)	Fall At (ns)	Add Clock	Source Objects	Source File	Scoped Cell	Current Instance
1		8.000			☐	[get_ports clk]	clk_PLL.xdc	PLL_1/inst	
3	sys_clk_pin	8.000	0.000	4.000	☒	[get_ports clk]	Cora-Z7-10-Master_TP5.xdc		

Double click to create a Create Clock constraint

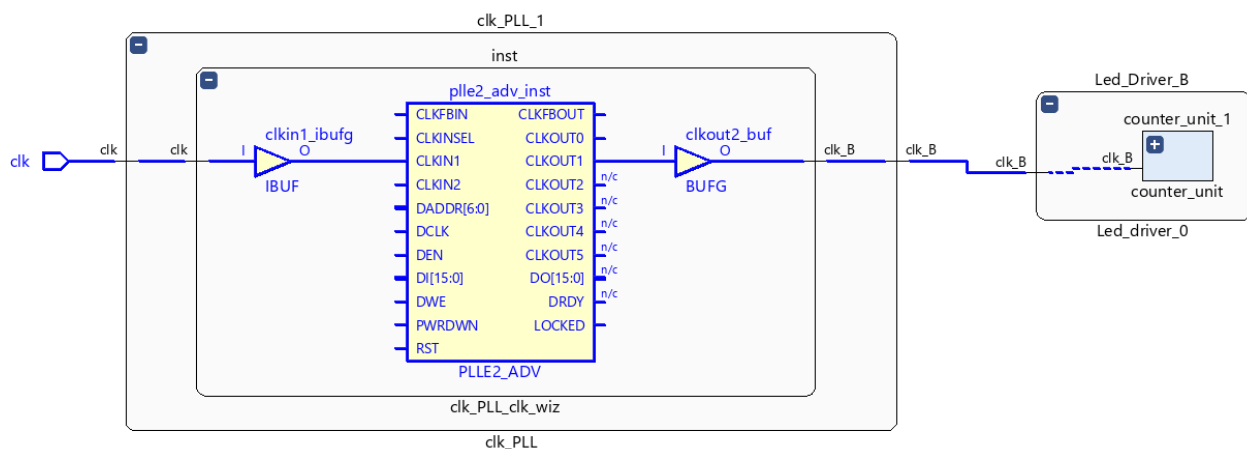
Le fichier de contrainte crée une horloge sans nom, connecté à la même source que l'horloge `Sys_clk_pin` déclarée dans notre fichier de contrainte. Les paramètres de cette horloge ne sont pas éditables.

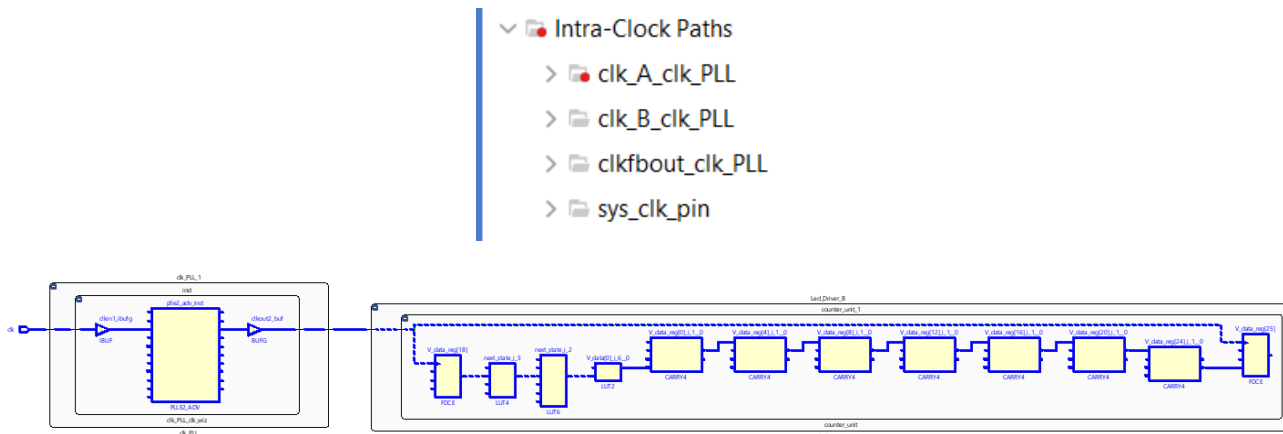
2 – Déclarer une seule horloge liée à `sys_clk_pin`.

Create Clock									
Position	Clock Name	Period (ns)	Rise At (ns)	Fall At (ns)	Add Clock	Source Objects	Source File	Scoped Cell	
1	sys_clk_pin	8.000	0.000	4.000	☒	[get_ports clk]	Cora-Z7-10-Master_TP5.xdc		



Le second problème est un erreur de **setup** dans le compteur de temporisation du driver de led régie par l'horloge la plus rapide.





La mise en cascade de 7 carry 4 ralentit le signal et provoque l'erreur de timing.

Le temps de propagation dans les carry 4 et les trois luts est supérieur à la période de l'horloge à 250MHz. D'après l'étude de timing, c'est la route qui prend le plus de temps. 70 %.

Pour résoudre le problème, il est possible de :

- 1/ Diminuer le cadencement du circuit, passer la fréquence d'horloge à 200Mhz.
- 2/ Définir manuellement des zones d'exclusion/d'implémentation des unités logiques.
- 3/ Diminuer les délais de temporisation pour réduire le nombre de bascules, Une fréquence de 250MHz peut admettre jusqu'à 26 bascules soit compter jusqu'à 67 000 000 coups d'horloges.

	25 bascules	26 bascules	27 bascules
Worst total delays (ns)	3,798	3,918 (+0,120)	4,181 (+0,263)
Requirement (ns)	4,0	4,0	4,0
Critical Slack (ns)	0,173	0,055	-0,209

$$F_{clk_a} = 250\text{MHz}$$

Démo avec une fréquence de 250Mhz et 26 bascules dans le fichier de rendu Cf TP5_demo_fast.mp4

- 4/ Modifier l'écriture du compteur de temporisation pour modifier le chemin critique.

Solution mis en œuvre	Slack à 250MHZ (ns)
Solution initiale.	- 0,412
Mémorisation de SV_fin_tempo_A dans fsm_led_driver.	- 0,167
Retrait du reset commentaire de la ligne 35 dans counter_unit (reset vide).	+ 0,317
Sortir Vdata ≤ Vdata+1 du if	- 0,412
Réécriture de la fonction avec mémorisation du vecteur data.	+ 0,013

```
architecture behavioral of counter_unit is
    -- Calcul de la taille de V_data avec la constante d'entrée.
    constant Cst_nb_basculer : integer := integer(ROUND(LOG(Cst_delai)/LOG(2.0)));
    constant Cst_taille_vector : integer := Cst_nb_basculer -1;
    --Declaration des signaux internes
    signal V_data : std_logic_vector (Cst_taille_vector downto 0):= (others => '0'); --signal interne s'incrémentant à chaque coup d'horloge
    signal V_cible : std_logic_vector (Cst_taille_vector downto 0); --Lorsque cette valeur est atteinte, on réinitialise le compteur.
    signal S_end_counter : std_logic:= '0'; -- signal interne de fin de compteur

begin
    V_cible <= std_logic_vector(to_unsigned(integer(Cst_delai),Cst_nb_basculer))- "1" ; --conversion du décimal vers un vecteur binaire.
    -- On enlève 1 car on compte à partir de 0

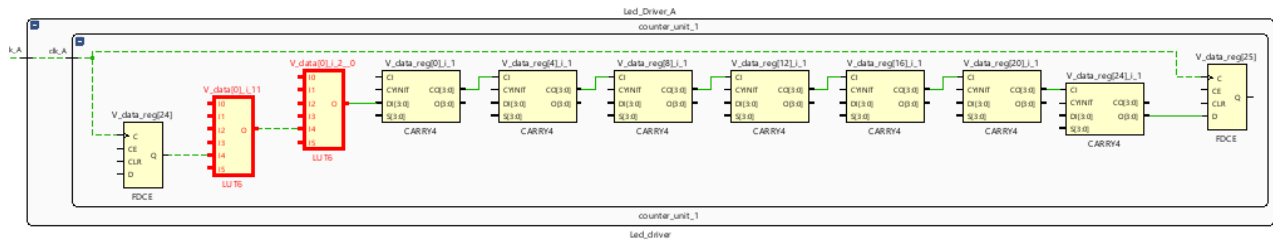
    --Partie sequentielle
    process(clk,resetn)
    begin
        if(resetn = '0') then
            --resetn Asynchrone
            V_data <= (others => '0');
        elsif(rising_edge(clk)) then
            if (S_end_counter = '1' or restartn = '0') then
                V_data <= (others => '0');
            else
                V_data <= V_data + "1";
            end if;
        end if;
    end process;

    --Partie combinatoire
    S_end_counter <= '1' when V_data > V_cible else '0';
    end_counter <= S_end_counter;
end behavioral;
```

Rédaction originale provoquant l'appartition de trois luts.

```
begin
    V_cible <= std_logic_vector(to_unsigned(integer(Cst_delai),Cst_nb_basculer))- "1" ;
    process(clk,resetn)
    begin
        if(resetn = '0') then
            --resetn Asynchrone
            V_data <= (others => '0');
        elsif(rising_edge(clk)) then
            if ((V_data_mem > V_cible - "1") or restartn = '0') then
                V_data <= (others => '0');
                S_end_counter <= '1';
            else
                V_data <= V_data + "1";
                S_end_counter <= '0';
            end if;
        end if;
    end process;
    V_data_mem <= V_data;
    end_counter <= S_end_counter;
end behavioral;
```

J'ai réussi à obtenir un slack positif avec cette rédaction, puis j'ai fait des modifications en dehors du counter_unit et le slack est repassé positif...



Après ré-écriture de compteur de temporisation, il est possible de retirer une LUT ce qui permet d'implémenter la solution à 250MHz.

La modification de l'horloge dans le fichier de contrainte modifie la valeur du requirement ce qui peut provoquer un slack positif mais uniquement lors de la compilation. Il faut se rappeler que la fréquence de la carte est fixée par le quartz.

11. Etudiez le rapport de synthèse.

Report du nombre de ressources utilisées,

Report Cell Usage:		
	Cell	Count
1	BUFG	3
2	CARRY4	14
3	LUT2	67
4	LUT3	8
5	LUT4	11
6	LUT5	7
7	LUT6	8
8	PLLE2_ADV	1
9	FDCE	76
10	FDPE	2
11	FDRE	2
12	LDC	4
13	LDCP	2
14	IBUF	2
15	OBUF	6

Commentaire du schéma RTL généré cf : TP5_Schematic_complet.pdf ou TP5_schematic.pdf

Logique de pilotage externe à led_driver et la PLL			
Nom de la ressource	Nb Bascules	Couleur	explication
<i>SV_nb_cycle</i>	5	Vert	Pour compter jusqu'à 20 on à besoin de 5 bascules.
<i>SV_nb_tempo_CDD</i>	4	Rouge	Pour compter jusqu'à 10 on à besoin de 4 bascules.

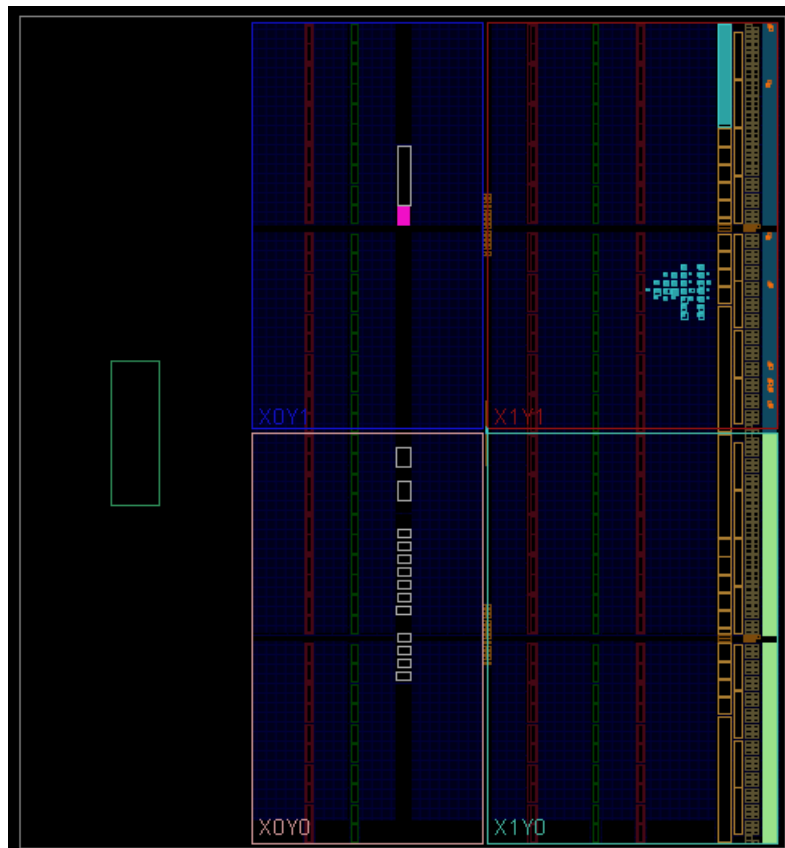
<i>FSM_Next_state</i>	4	Magenta	3 bascules FDCE, 1 bascule FDPE.
<i>FSM_current_state</i>	4	Cyan	3 bascules FDCE, 1 bascule FDPE.
<i>Update</i>	1	violet	Synchronisation en sortie de l'unité logique.
<i>Update_decal</i>	1	Orange	Bascule de temporisation du signal
Utilisation des ressources pour Led_driver			
Nom de la ressource	Nb Bascules	Couleur	explication
<i>S_color_MEM</i>	2	Vert	Il y a 3 valeurs possibles pour <i>S_color_code</i> donc 2 bascules.
<i>Next_state_red and current_state_reg.</i>	2	Rouge	Machine à Etat d'allumage et d'extinction des leds.
<i>Bascule des états des leds en sortie</i>	3	Bleu	
Utilisation des ressources pour Compteur de temporisation			
Nom de la ressource	Nb Bascules	Couleur	explication
<i>V_data_reg</i>	27	Rouge	27 Bascules permettent de compter jusqu'à 134 217 728. c'est suffisant pour les 100 000 000 coups d'horloge requis.

FDPE : D-type flip-flop with asynchronous Preset. Dans notre cas, l'entrée *Preset* est connectée à la porte logique ou prenant en entrée le *reset_general* et la sortie *locked* de le PLL. Cette bascule permet donc de se placer dans l'état *init*.

LUT : *FSM_onehot_current_state[3]_i_1* → LUT Lut de gestion des *resets*, permet le maintien du *reset* tant que *locked* est à l'état bas.

LUT : *FSM_onehot_next_state[3]_i_1* → LUT 2 luts pour la gestion des états de la machine d'état.

12. Effectuez le placement routage et étudiez les rapports générés.



Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 0,013 ns	Worst Hold Slack (WHS): 0,188 ns	Worst Pulse Width Slack (WPWS): 1,500 ns
Total Negative Slack (TNS): 0,000 ns	Total Hold Slack (THS): 0,000 ns	Total Pulse Width Negative Slack (TPWS): 0,000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 88	Total Number of Endpoints: 88	Total Number of Endpoints: 87

All user specified timing constraints are met.

Les contraintes de timing sont respectées, les slacks sont positifs. Le chemin critique est celui mis en évidence plus haut dans le rapport, il passe par le compteur de temporisation et notamment dans les sommateurs requis pour incrémenter le vecteur V_data .

La plus grande taille de la fenêtre : 131072 sample, le cadencement de l'ILA utilise la fréquence à 150MHz, la profondeur temporelle est donc égale à $131072 * (1/150\text{MHz}) = 874$ microsecondes.

Fréquence d'horloge A : 150 MHz Fréquence d'horloge B : 50MHz

Delai de temporisation : 100 000 000 coup d'horloge avec deux temporisations pour un clignotement. La durée d'un clignotement est égale à 1,3s pour l'horloge A et 4s pour l'horloge B.

Il faut donc baisser la fréquence de clignotement pour l'observer à l'ILA.

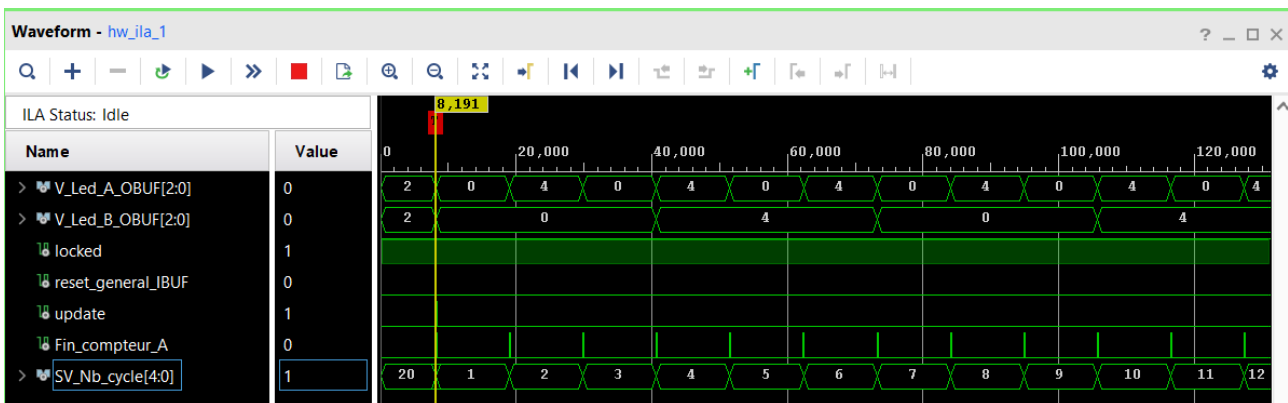
Admettons que je veuille capter deux clignotements à 50Mhz, quel est le nombre de coup d'horloge qu'il faut compter ?

$$4 \times n_{tempo} \times t_B = nb_{sample} \times t_{ila} \Rightarrow n_{tempo} = \frac{nb_{sample} \times t_{ila}}{4 \times t_B} \Rightarrow \frac{131072 \times 50 \times 10^6}{4 \times 150 \times 10^6} = 10922$$

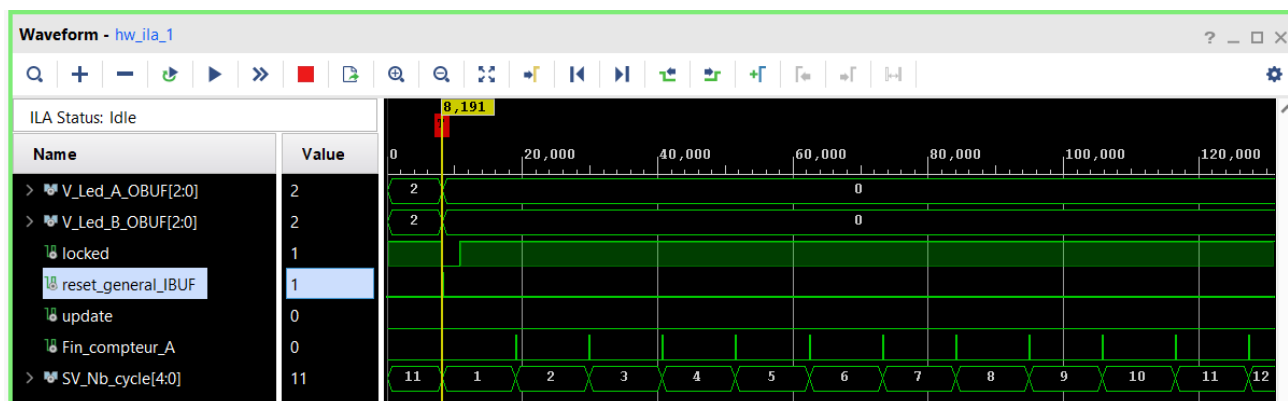
Empiriquement, j'ai trouvé qu'il fallait fixer le délais de temporisation à 43600 coups d'horloges.

Pourquoi une telle différence ?

The debug hub is responsible for the communication between Vivado IDE and the debug cores (ILA and VIO). We see that it defines a clock frequency (default is 300 MHz). You need to change that clock to match your clock frequency and save the file.
Donc 300Mhz n'est pas la fréquence d'échantillonnage.



Le changement de fréquence est bien remarquable sur la capture ci-dessus. Le changement de couleur s'effectue lorsque *Update* passe à l'état haut. *SV_nb_cycle* s'incrémente à chaque *fin_compteur_A*.



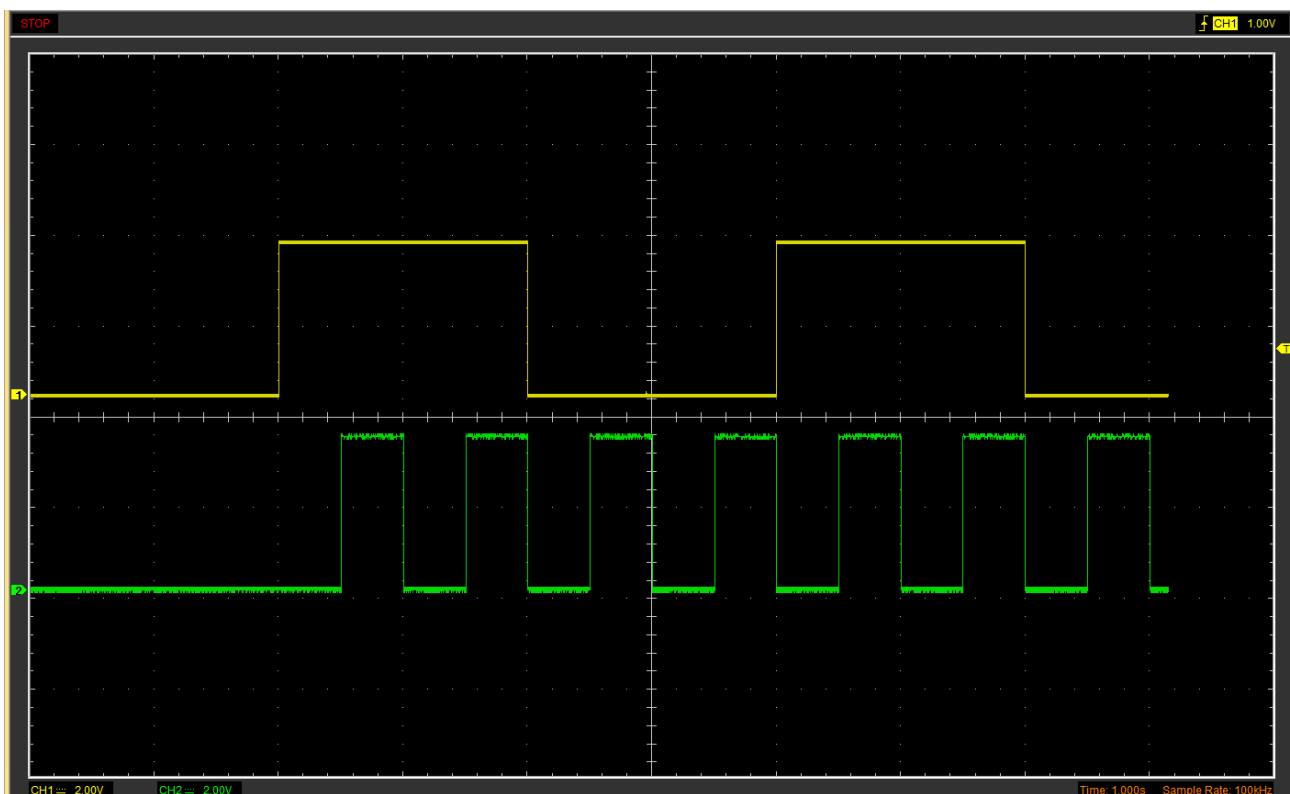
Sur pression du *reset*, *SV_Nb_Cycle* est réinitialisé à 0, la PLL est *reset* provoquant le mis à l'état bas de *locked* le temps que les horloges se stabilisent. Les diodes ne clignotent pas, cela indique que la machine à état est dans l'état *init*.

13. Générez le bitstream, programmez la carte et vérifiez les signaux du chipscope (ILA).

Cf : Demo_TP5_Q13

On observe que les deux diodes clignent à des fréquences différentes. Les diodes qui clignent rapidement sont pilotées par l'horloge A, elles clignent 10 fois. Celles qui clignent plus lentement clignent moins de fois car c'est les clignotement de led driver A qui comptent.

Après un état d'initialisation, elles clignent suivant la séquence rouge, bleu, vert.



Mesure de la fréquence de chaque diode à l'oscilloscope $1 \text{ s}/\Delta V$.

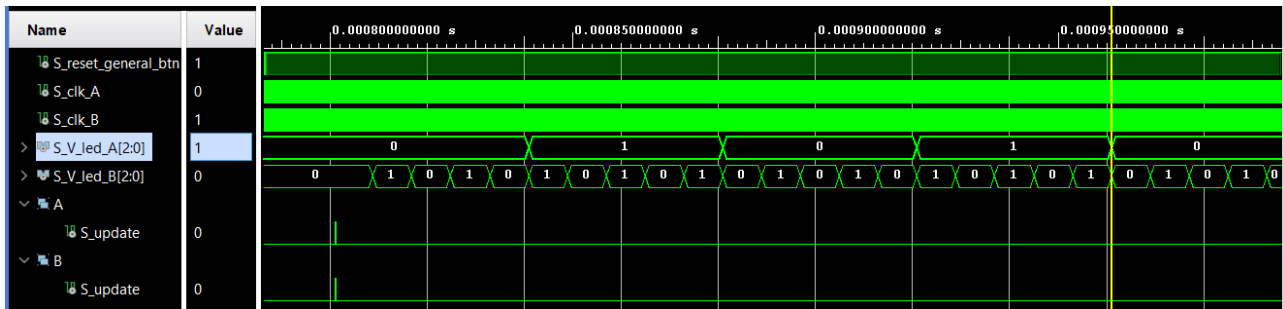
Configuration : Fréquence horloge A : 200MHz, Fréquence horloge B : 50Mhz, temporisation 100 000 000 périodes.

Période de clignotement horloge A : 4s.

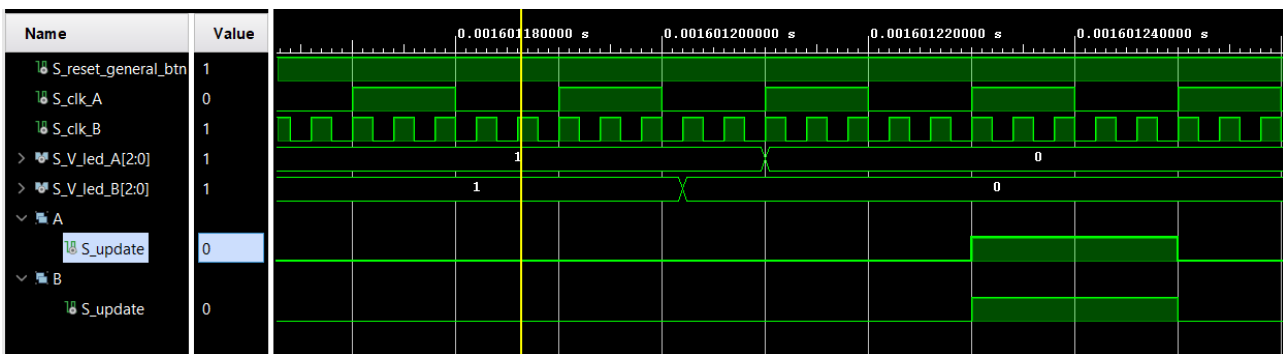
Période de clignotement horloge B : 1s.

14. Imaginez une solution de passage à un domaine de fréquence Lent vers rapide.

Cette fois-ci, c'est l'horloge lente cadence le système et un led driver. Deux problèmes surviennent :



le premier problème survient car le signal d'actualisation des deux drivers utilise la fin de temporisation du module piloté par le signal d'horloge de fréquence la plus basse, les leds pilotées par le driver B clignotent plus de 10 fois.

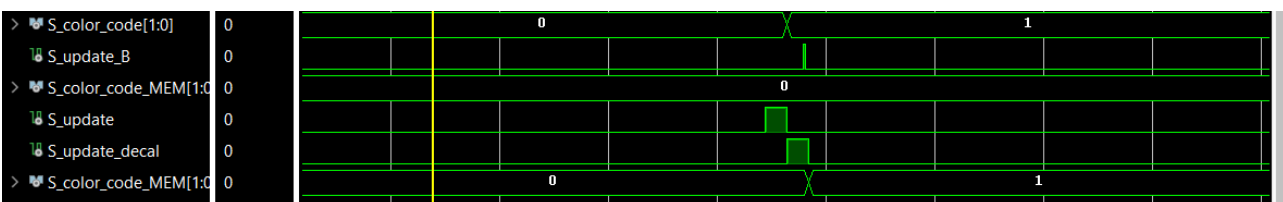


Le second problème est que la durée du signal d'update pour le driver B dure plus d'un coup d'horloge. Ce qui ne provoque pas de dysfonctionnement logique mais qui n'est pas conforme au résultat attendu.

Deux bascules sont ajoutées pour détecter le front montant d'update.

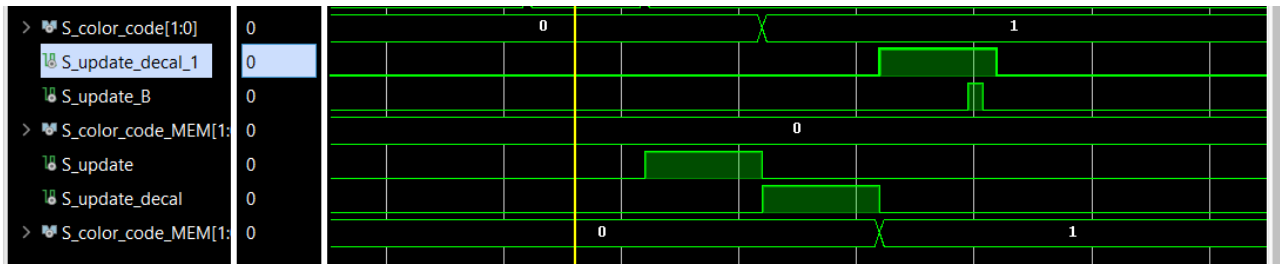
Cf Schéma RTL TP5_Q14.png

Le signal *d'update* entre dans la machine à état, il faut un coup d'horloge pour que *color_code* s'actualise, c'est la raison pour laquelle j'ai utilisé une bascule pour décaler le signal *d'update* d'un coup d'horloge pour alimenter *led_driver* cadencé par l'horloge A avec *update_decal*. Lorsque le front montant de ce signal est utilisé pour générer *Update_B*, le signal dédié au *led_driver* cadencé par l'horloge B, *color_code* n'est jamais pris en compte. Cf chronogramme ci-dessous.

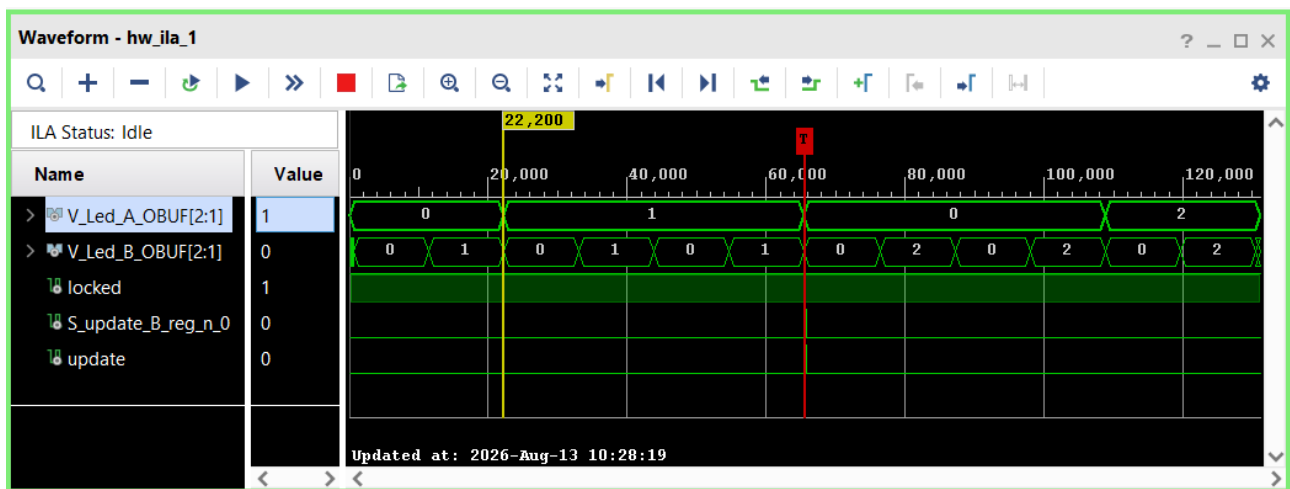


Bien que *S_color_code* change au front descendant de *S_update* et que *S_update_B* survient après ce changement, *S_color_MEM* de led driver B n'est jamais pris en compte. Alors que le vecteur de mémorisation de la couleur est correctement mis à jour pour *led_driver_A*.

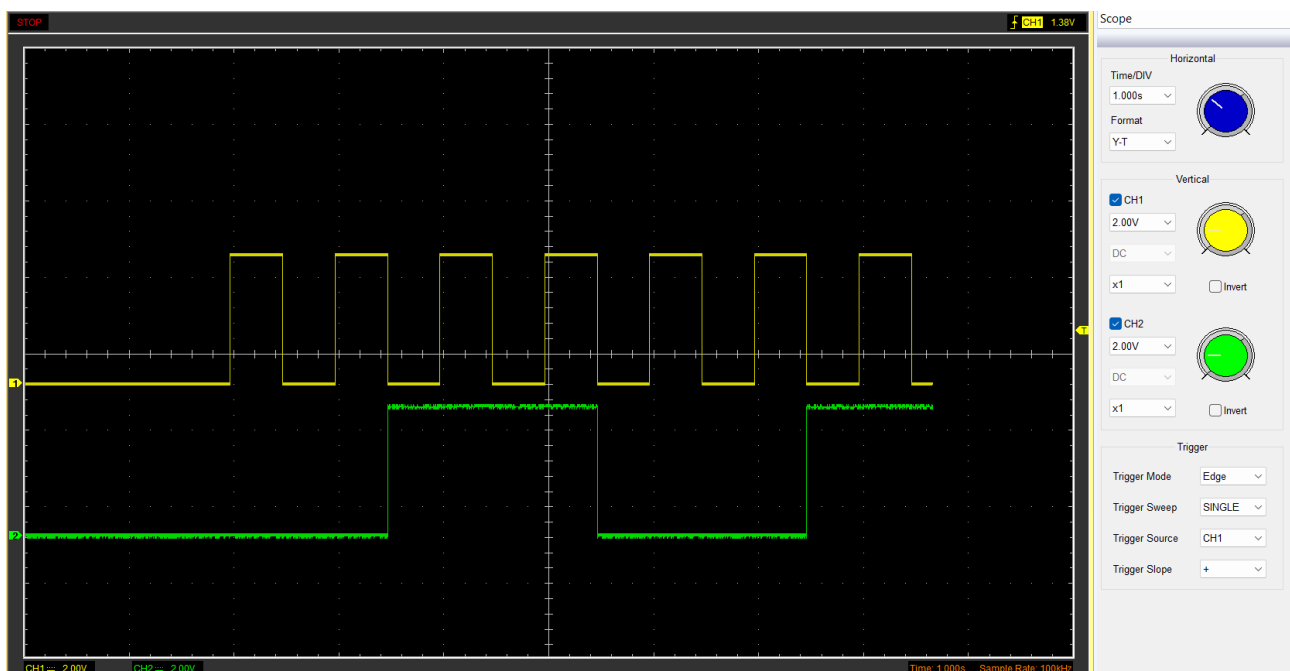
En décalant de nouveau le signal *d'update*, la solution choisie fonctionne correctement.



A l'ila on constate bien le changement de fréquence d'horloge. Cette fois-ci ce sont les leds A qui clignotent plus lentement que les leds B. Comme c'est toujours les 10 clignotements des leds A qui comptent, les leds B clignoteront plus de 10 fois.



On constate également que `S_update_B` dure moins longtemps que `S_update`.



Cette fois-ci les fréquences sont inversées, Le Chanel 1 connecté à la diode rouge pilotée par *led_driver* cadencé par l'horloge B est plus rapide que le Chanel 2 qui est relié à l'horloge A.

La période de clignotement est égale à 2s pour le chanel le plus lent et 0,5s pour le plus rapide conformément aux fréquences de fonctionnement exigées par la PLL (200MHz et 50MHz).

15. Modifiez votre solution afin que les leds ne clignent que 10 fois.

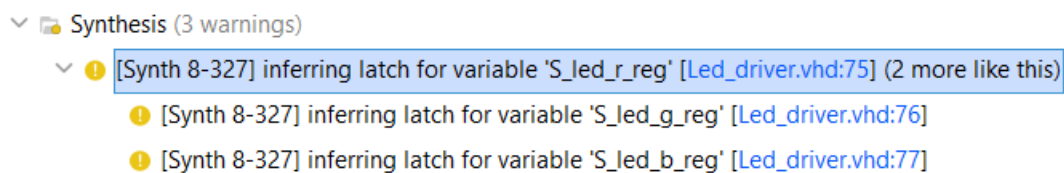
Schéma rtl cf Q15 :

Démo : TP5_Q15.

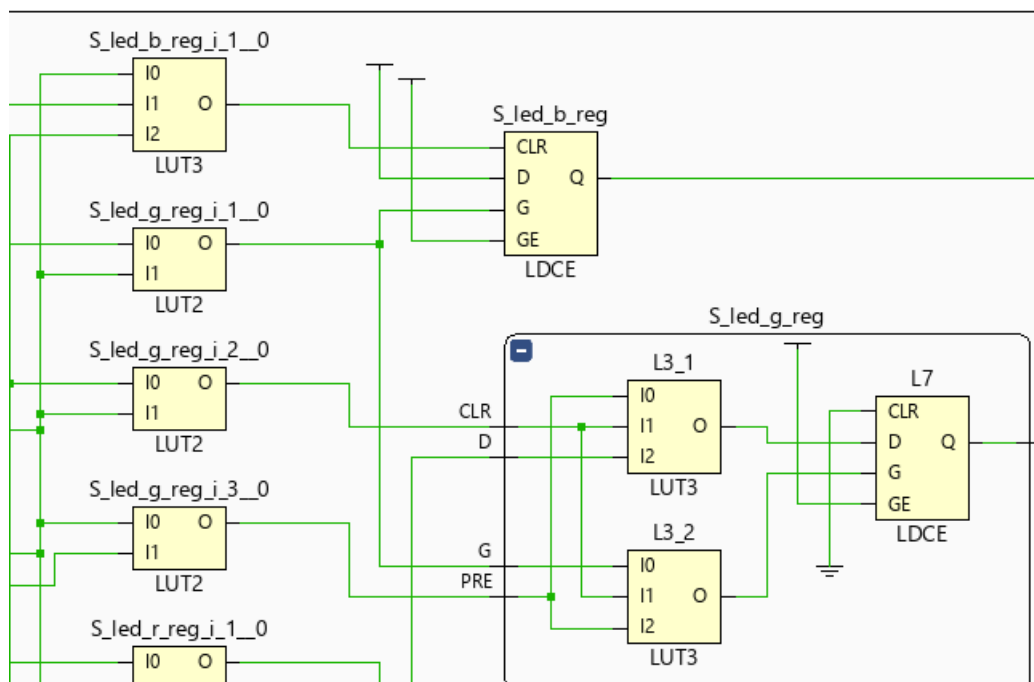
De manière similaire à la solution mis en place à la question 13, on initialise une variable qui va être décompté pour limiter le nombre de clignotement de la led piloté par l'horloge de plus haute fréquence.

Un signal *Enable* est aussi ajouté dans *led_driver* pour éteindre les diodes. Le signal *d'update* à été décalé d'un coup d'horloge pour permettre l'actualisation du signal *d'Enable*.

---- Résolution des warnings



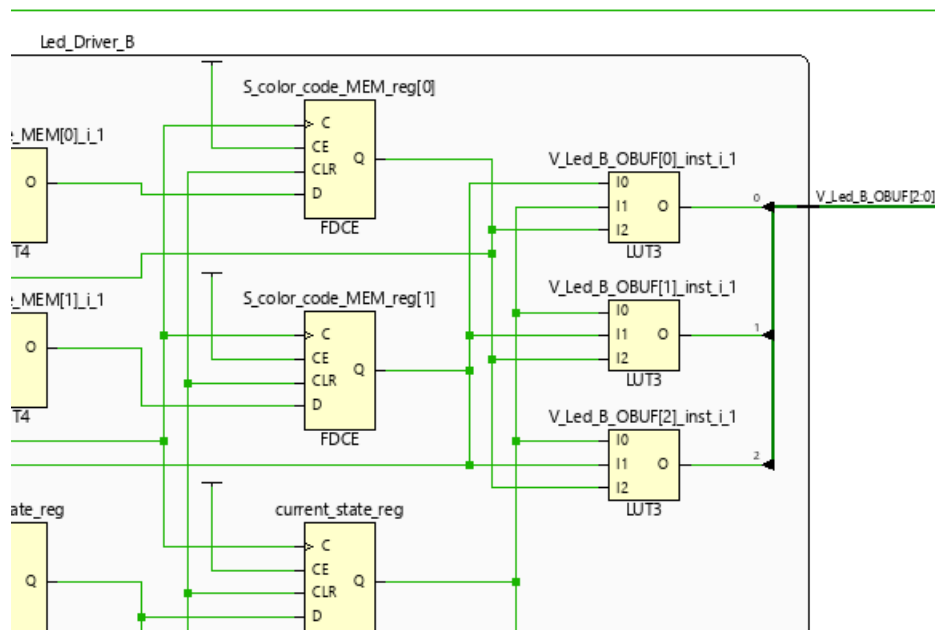
Vivado indique qu'une latch à été inférée pour la variable *S_led_r_reg*. C'est une latch LDCE (Cf TP4) **Asynchrone** qui se trouve en sortie de la logique combinatoire permettant la sélection des couleurs.



L'entrée *Clear* est connectée à une lut, *Gate enable* et *D* toujours à l'état haut. La latch s'actualise sur front montant du signal de *Gate* qui est une sortie de la logique combinatoire de sélection des couleurs. Donc c'est toujours un 1 qui recopié sur *Q*, puis maintenu jusqu'à un changement de couleurs.

L'IDE lève un warning car les délais de propagation des latches ne peuvent pas être mesurés lors de l'analyse de timing et risque de provoquer des problèmes lors de l'implémentation.

La latch est inférée car tous les états des variables ne sont pas définis, ajouter le *when other* du *case* statement ainsi que le *else* du *if* dans la description de *led_driver* enlève les latches.



Rq suite à la correction du TP3

- Ajout du restartn, J'ai choisi un restart synchrone et un reset asynchrone.
Il est plus logique de le faire dans ce sens car le reset peut être souvent matériel (bouton sur la carte)

La liste de sensibilité :
d'un process **synchrone** prend en argument le **reset** (asynchrone) et **l'horloge**.
Dans un process **asynchrone** c'est tous les signaux qui sont lus.